Data Dynamo Project Team 2 (FROM)

By Alice, George, Luqmaan & Tawheed

Section 1: Introduction & Setup

1.1 Data Preparation Notes

The first issue we ran into was before saving the spreadsheet as a .csv file. The column names were in invalid formatting for tabular data. Some were hyphenated, some had spaces, some had underscores and casing wasn't consistent. Therefore, we edited the column names. We changed them all to lowercase with underscores replacing spaces before saving the file as a .csv file.

We initially set VARCHAR limit for product name to 100 but data exceeded limit even after increasing to 200. To solve this, we removed limit on VARCHAR. However, if you want to use limits, we have found that the ones to specifically change are publication and product name which need a limit of more than 58 and 154 respectively.

1.2 Architecture Overview

During ingestion and querying, the following issues were identified:

• Boolean values imported inconsistently (0/1 vs TRUE/FALSE)

During the creation phase, initially the BOOL data type was assigned to two columns (special_day & online_sale_offers). But during the import, the values were inconsistent with the source data.

To address these issues, a two-layer architecture was implemented:

Excel / CSV

↓

Prep Table (raw types, minimal transformation)

↓

Final Table (altered, analysis-ready schema)

Table Design

This table preserved raw values while allowing controlled transformations later.

```sql
-- STEP 1
-- Creating the table
CREATE TABLE data_dynamo.mm_books (
    id INT,
    purchase_date VARCHAR,
    special_day INT,
    online_sales_offer INT,
    customer_id VARCHAR,
    gender VARCHAR(1),
    product_name VARCHAR,
    item_status VARCHAR,
    quantity INT,
    currency VARCHAR,
    item_price INT,
    shipping_price INT,
    ship_city VARCHAR,
    ship_state VARCHAR,
    ship_postal_code INT,
    category varchar,
    total_amount INT,
    author VARCHAR,
    publication_ VARCHAR,
    profit_percentage INT,
    profit NUMERIC,
    cost_price NUMERIC
);
```

Rationale:

- Dates and flags are ingested as text/integers to avoid import-time failures.
- No implicit casting is relied upon.

The Booleans and flags were then converted using an ALTER query.

<u>Final Alterations</u>

```sql
-- STEP 2
-- Alter tables to turn special_day and online_sales_offer into BOOL types
ALTER TABLE data_dynamo.mm_books
    ALTER COLUMN special_day TYPE BOOL
    USING CASE
        WHEN special_day = 1 THEN TRUE
        WHEN special_day = 0 THEN FALSE
        ELSE NULL
    END;

ALTER TABLE data_dynamo.mm_books
    ALTER COLUMN online_sales_offer TYPE BOOL
    USING CASE
        WHEN online_sales_offer = 1 THEN TRUE
        WHEN online_sales_offer = 0 THEN FALSE
        ELSE NULL
    END;


-- STEP 3
-- Alter table to turn purchase date into a timestamp
ALTER TABLE data_dynamo.mm_books
    ALTER COLUMN purchase_date TYPE TIMESTAMP
    USING TO_TIMESTAMP(purchase_date, 'DD/MM/YYYY HH24:MI');
```

Section 2: Data Import Verification

2.1 Import Validation

We have checked the import is consistent with the data from the original spreadsheet and that the data is the same in both. Here are a few queries we used to test this:

Customer_ID:

VALUE	FREQUENCY
B07FWVUVY	9
B07FWVUUU	8
B07FWVUWS	7
B07FWVUVZ	7
B07FWVUTT	6

```sql
SELECT
    COUNT(mb.customer_id),
    mb.customer_id
FROM data_dynamo.mm_books mb
GROUP BY mb.customer_id
ORDER BY 1 DESC
LIMIT 5;
```

	123 count	A-Z customer_id
1	9	B07FWVUVY
2	8	B07FWVUUU
3	7	B07FWVUWS
4	7	B07FWVUVZ
5	6	B07FWVUUX

Gender:

VALUE	FREQUENCY
M	1,815
F	1,665

```sql
--Gender validation

SELECT
    COUNT(mb.gender),
    mb.gender
FROM data_dynamo.mm_books mb
GROUP BY mb.gender
ORDER BY 1 DESC;
```

	123 count	A-Z gender
1	1,815	M
2	1,665	F

Category:

VALUE	FREQUENCY
school	1,277
competition	854
college	761
School	588

```sql
--Category validation

SELECT
    COUNT(amb.category),
    amb.category
FROM data_dynamo.mm_books amb
GROUP BY amb.category
ORDER BY 1 DESC;
```

	123 count	A-Z category
1	1,277	school
2	854	competition
3	761	college
4	588	School

Section 3: Data Quality Analysis

3.1 Completeness Assessment

Regarding completeness of the dataset, we have not found any null/ missing values:

```sql
-- Check for NULL values

SELECT
    COUNT(*) - COUNT(id) AS id_nulls,
    COUNT(*) - COUNT(special_day) AS special_day_nulls,
    COUNT(*) - COUNT(online_sales_offer) AS online_sales_offer_nulls,
    COUNT(*) - COUNT(quantity) AS quantity_nulls,
    COUNT(*) - COUNT(item_price) AS item_price_nulls,
    COUNT(*) - COUNT(shipping_price) AS shipping_price_nulls,
    COUNT(*) - COUNT(ship_postal_code) AS postal_nulls,
    COUNT(*) - COUNT(total_amount) AS total_amount_nulls,
    COUNT(*) - COUNT(profit_percentage) AS profit_pct_nulls,
    COUNT(*) - COUNT(profit) AS profit_nulls,
    COUNT(*) - COUNT(cost_price) AS cost_price_nulls
FROM data_dynamo.mm_books;
```

Results 1

SELECT COUNT(*) - COUNT(Enter a SQL expression to filter results (use Ctrl+Space)

	id_nul	speci	online_	quant	item	shippin	post	tota
1	0	0	0	0	0	0	0	0

3.2 Uniqueness Assessment

Regarding uniqueness of the dataset, we have not found any duplicate records:

```sql
select mm.purchase_date, mm.customer_id, COUNT(*) from data_dynamo.mm_books mm
group by mm.purchase_date, mm.customer_id
having COUNT(*) > 1;
```

purchase_date	customer_id	count

3.3 Timeliness Assessment

Regarding timeliness of the dataset, we have queried the database for the oldest and newest purchase:

The data provided starts from the 1st of January 2018 and goes up to the 31st of December 2019. The year is currently 2026 so we leave it up to the discretion of Mystic Manuscripts to decide if this data is timely enough for them.

```sql
--Max and Min Dates (Timeliness check)

SELECT
    MIN(amb.purchase_date) Oldest,
    MAX(amb.purchase_date) Newest
FROM data_dynamo.mm_books amb;
```

	oldest	newest
1	01/01/2018 08:57	31/12/2019 12:02

Section 4: Data Quality Issues Identified

4.1 Validity Issues

4.1.1 Inconsistent Text Formatting

The first query below returned 4, whereas the second returned 3, which confirms that the categories were not formatted correctly (school vs. School) which are categorically the same but is taken as two separate categories in SQL.

```sql
SELECT COUNT(DISTINCT amb.category) unique_categories
FROM data_dynamo.mm_books amb;

SELECT COUNT(DISTINCT LOWER(amb.category)) unique_categories
FROM data_dynamo.mm_books amb;
```

There are a few columns that aren't well regulated. For example, there are some authors whose names are recorded slightly differently in different places. The example below is of an author called "Kanti Jain". But there are many variations on the name including having a space at the beginning, having a space at the end and being added in to different groups and lists.

	count	author
1	246	MAHAVEER JAIN,ANSHUL JAIN KANTI JAIN
2	72	Kanti Jain
3	58	Kanti Jain
4	16	kanti jainmahiveer jainAnshul jain
5	3	KANTI JAIN, MAHAVEER JAIN¶
6	3	Kanti Jain, Mahaveer Jain, Anshul Jain
7	2	ANSHUL JAIN, KANTI JAIN, MAHAVEER JAIN
8	2	ansul jain, kanti jain
9	2	Kanti Jain

This makes it difficult to find a specific author in the database and leads to confusion when trying to query the database.

The columns that have this problem include Authors, Publication, Ship City, Ship State and Category.

4.1.2 Whitespace Issues

When investigating the data, whitespace issues were discovered in many of the text columns, we checked product_name, category, author, and publication columns for leading/trailing whitespace.

The results showed 854 rows have whitespace (~ 25%) in the category column, 87 in author, and 5 in publication. Product_name however looked clean. These whitespace issues need to be cleaned as they will cause problems with filtering, grouping and matching operations.

```sql
-- Whitespace Check

SELECT
    COUNT(CASE
        WHEN product_name != TRIM(product_name) THEN 1
        END) product_name_whtspce,
    COUNT(CASE
        WHEN category != TRIM(category) THEN 1
        END) category_whtspce,
    COUNT(CASE
        WHEN author != TRIM(author) THEN 1
        END) author_whtspce,
    COUNT(CASE
        WHEN amb.publication_ != TRIM(amb.publication_ ) THEN 1
        END) publication_whtspce
FROM data_dynamo.mm_books amb;
```

	product_name_whtspce	category_whtspce	author_whtspce	publication_whtspce
1	0	854	87	5

The code below does not use the usual TRIM() function because the spaces that are in some columns (such as the author column) are “no-break spaces” and so cannot be trimmed using TRIM().

```sql
update data_dynamo.mm_books_prep
set author = lower(btrim(replace(author, E'\u00A0', '')));
```

4.2 Calculation Inconsistencies

4.2.1 Profit Calculation Mismatches

The profit column should be calculated as total_amount – cost_price – shipping_price. This is the case for most rows, however for some, the profit value is slightly off, even when there are no offers available (online_sales_offer = 0).

Special days don't seem to make a difference regarding prices and profits, and no offers are used in these queries, so there is a discrepancy here.

```sql
--Expected profits (Profit = total_amount - cost_price - shipping_price)

SELECT
    id,
    profit,
    (total_amount - cost_price - shipping_price) expected_profit
FROM
    data_dynamo.mm_books amb
WHERE
    profit != (total_amount - cost_price - shipping_price)
    AND online_sales_offer IS FALSE;
```

4.2.2 Total Amount Calculation Mismatches

Total amount for each row has mismatches - we tried to check the total_amount various ways, all of which show discrepancies.

```sql
-- Check if total_amount = (item_price * quantity) + shipping_price
SELECT
    COUNT(*) calculation_mismatch
FROM data_dynamo.mm_books amb
WHERE ABS(amb.total_amount - ((amb.item_price * amb.quantity) + amb.shipping_price )) > 0.01
    AND amb.online_sales_offer IS FALSE;

-- Check if total_amount = item_price + shipping_price
SELECT
    COUNT(*) calculation_mismatch
FROM data_dynamo.mm_books amb
WHERE ABS(amb.total_amount - ((amb.item_price) + amb.shipping_price )) > 0.01
    AND amb.online_sales_offer IS FALSE
    AND amb.special_day IS FALSE;
```

• First, we used the calculation total_amount = (item_price * quantity) + shipping_price, which when discounting any days with sales offers returned 153 discrepancies.

123 calculation_mismatch
153

• Then we tried without multiplying the quantity, as we hypothesized this may have already been done in the table, which returned 58 mismatches (48 if special days were also excluded).

123 calculation_mismatch
58

Because both calculations showed major discrepancies, meaning that it makes sense to assume there have been calculation errors however the total amount was intended to be calculated. Therefore, a single formula needs to be decided on and properly implemented across the dataset.

4.3 Data Consistency Validation

We computed the sum of row sums and sum of column sums for all numeric columns and obtained 1,474,266,021.7900 for both. This is a mathematical expectation since both calculations compute the same value, the grand total. We validated this by testing for NULL values using SQL queries and the COALESCE function, verifying individual column sums, and checking data integrity across multiple methods, all of which consistently produced the same result.

While this differs from the provided expected answers, our approach and validation demonstrate the logical soundness of 1,474,266,021.7900 as the correct value; as the only time they should have a difference if and only if there are null values in at least one of the numerical columns.

Section 5: Recommendations

5.1 Proposed Data Standardisation

5.1.1 Case Standardisation

This is an update to columns to fix the problem with there being multiple values that are the same but because they have differences in case, they are returned as separate values.

This change is suggested for the Authors, Publication, Ship City, Ship State and Category columns. It isn't necessary for the Product Name and Item Status as they are, however, you could also do this for futureproofing.

```
update data_dynamo.mm_books
set category = lower(category);

update data_dynamo.mm_books
set product_name = lower(product_name);

update data_dynamo.mm_books
set item_status = lower(item_status);
```

5.1.2 Whitespace Removal

There is also a problem where spacing before and/or after a value causes the same issue, so we also recommend trimming the ends of each value:

```
update data_dynamo.mm_books
set author = lower(btrim(replace(author, E'\u00A0', '')));
```

5.1.3 Column limits

Remove manually set limits for VARCHAR data types – VARCHAR(100) becomes VARCHAR. If you want to use manual limits, then publication should have a limit that is more than 58 and product_name should have a limit that's more than 154.

5.1.4 Boolean values

Alter the table so that the special_day and online_sales_offers columns have data type BOOL, and convert the values accordingly (0 to false, 1 to true).

Note: When calculating the sum of the numeric values, special_day and online_sales_offers should be converted to integers.

Conclusion

After implementing the changes suggested in this report, the dataset should be ready for further use.